COURSE CODE/TITLE:	CPE10- Data Structures and Algorithm	**SECTION:**	CPE21S4
DATE PERFORMED:	8/8/26	**DATE SUBMITTED:**	8/8/26
NAME:	Peñaflorida,Wrio Carlo M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Activity No. 5: Queues

1. PROCEDURE OUTPUT

ILOA:

Task: Create C++ code using Queue STL that will pass an array of students' names to the queue. Make

observations on the operations performed and show screenshots of code and output console per

operation.

main.cpp

```cpp
#include <iostream>
#include <queue>
#include <string>

void display(std::queue<std::string> q)
{
    std::queue<std::string> c = q;

    while (!c.empty())
    {
        std::cout << " " << c.front();
        c.pop();
    }
    std::cout << "\n";
}

int main()
{
    std::string students[] = {"Christine", "Ben", "Isabelle", "Diego", "Ella"};
    int n = sizeof(students) / sizeof(students[0]);

    std::queue<std::string> studentQ;

    // enqueue
    for (int i = 0; i < n; i++)
    {
        studentQ.push(students[i]);
    }

    std::cout << "The student queue is :";
    display(studentQ);

    std::cout << "studentQ.empty() : " << studentQ.empty() << "\n";
    std::cout << "studentQ.size()  : " << studentQ.size() << "\n";
    std::cout << "studentQ.front() : " << studentQ.front() << "\n";
    std::cout << "studentQ.back()  : " << studentQ.back() << "\n";

    std::cout << "After studentQ.pop() : ";
    studentQ.pop();
    display(studentQ);

    studentQ.push("Farah");
    std::cout << "After studentQ.push(\"Farah\") : ";
    display(studentQ);
```

```
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\HOA5\ILOB> g++ -g -Wall main.cpp -o main; if ($?) { .\main.exe }
The student queue is : Christine Ben Isabelle Diego Ella
studentQ.empty() : 0
studentQ.size()  : 5
studentQ.front() : Christine
studentQ.back()  : Ella
After studentQ.pop() :  Ben Isabelle Diego Ella
After studentQ.push("Farah") :  Ben Isabelle Diego Ella Farah
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\HOA5\ILOB>
```

ILOB:

Your Tasks:

- Create a queue class that will contain the array.
- Operations and member functions described in this section must be in your developed code.
- Test the functions and show the output.

main.cpp | queueLL.h | queueArr.h

main.cpp > main()

```cpp
#include <iostream>
#include "queueLL.h"
#include "queueArr.h"

int main() {

    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    enqueue(1, &front, &back);
    std::cout<< "front: " << front->data <<" back: "<< back->data<<"\n";
    enqueue(2, &front, &back);
    enqueue(3, &front, &back);
    std::cout<< "front: " << front->data <<" back: "<< back->data<<"\n";

    displayQueue(front);
    dequeue(&front, &back);
    std::cout<< "front: " << front->data <<" back: "<< back->data<<"\n";
    dequeue(&front, &back);
    std::cout<< "front: " << front->data <<" back: "<< back->data<<"\n";
    dequeue(&front, &back);
    dequeue(&front, &back);
    displayQueue(front);



    //object creation
    queueArr<int> q(5);

    q.Enqueue(1);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size: " << q.Size() << "\n";
    q.Enqueue(2);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size: " << q.Size() << "\n";
    q.Enqueue(3);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size: " << q.Size() << "\n";
    q.Enqueue(4);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size: " << q.Size() << "\n";
    q.Enqueue(5);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size: " << q.Size() << "\n";
    q.Enqueue(6);
    std::cout<< "front: " << q.front() <<" back: "<< q.back()<< " size: " << q.Size() << "\n";

    return 0;
}
```

main.cpp | queueLL.h | queueArr.h

queueLL.h > dequeue<T>(qNode<T>**, qNode<T>**)

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H
#include <iostream>

template <typename T>
class qNode {
public:
    T data;
    qNode<T> *next;
};

//creating the new node
template <typename T>
qNode<T> *new_node(T newDATA){
    qNode<T> *newNode = new qNode<T>;
    newNode->data = newDATA;
    newNode->next = NULL;
    return newNode;
}

template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
   //create a node node;
   qNode<T>* newNode = new_node(newData);

   //inserting into an empty queue
   //how do we know that the queue is empty?
   if((*frontPtr)== nullptr && (*backPtr)== nullptr){
      //assign the frontPtr to the newNode
      (*frontPtr)= newNode;
      //assign the backPtr to the newNode
      (*backPtr)= newNode;
      return;
   }
    //point the back next to the newNode:
    (*backPtr)->next = newNode;
    // the newNode will be the new backPtr'
    (*backPtr) = newNode;
    return;
}

template <typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr){
    //how to determine if the queue is empty?
    if((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        std::cout<< "The queue is empty. Nothing to dequeue.\n";
        return;
    }

    //creating temporary pointer:
    qNode<T>* tempPtr = nullptr;
    //store the frontptr to the temptr;
    tempPtr = (*frontPtr);

    //check if queue has only 1 node left
    if ((*frontPtr) == (*backPtr)){
        (*frontPtr) = nullptr;
```

```
54
55      //check if queue has only 1 node left
56      if ((*frontPtr) == (*backPtr)){
57          (*frontPtr) = nullptr;
58          (*backPtr) = nullptr;
59          delete tempPtr;
60          return;
61      }
62
63      //update the frontptr
64      (*frontPtr) = (*frontPtr)->next;
65      //point the temptr -> next to null
66      tempPtr->next = nullptr;
67      delete tempPtr;
68  }
69
70  //display all items the queue: check the traverse in singlyLL
71  template <typename T>
72  void displayQueue(qNode<T> *frontPtr){
73      //check if the queue is empty
74      if(frontPtr == nullptr){
75          std::cout<< "The queue is empty.\n";
76          return;
77      }
78      //temp pointer to store the front Ptr
79      qNode<T>* current = frontPtr;
80
81      //create a loop that will print the tempPtr->data
82      while(current != nullptr){
83          std::cout<< current->data << " ";
84          current = current->next;
85      }
86      std::cout << std::endl;
87  }
88
```

queueArr.h > ~queueArr()

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>


template <typename T>
class queueArr{
    private:
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;
    public:
        //constructor
        queueArr(size_t capacity = 10);

        bool Empty() const;
        size_t Size() const;
        void Clear();

        T front() const;
        T back() const;

        void Enqueue(T value);
        void Dequeue();

        //copy constructor:
        queueArr(const queueArr& other);

        //copy assignment operator:
        queueArr& operator=(const queueArr& other);

        //destructor:
        ~queueArr();
};

//constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity){
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}

template <typename T>
bool queueArr<T>::Empty() const{
    return (q_size == 0);
}

template <typename T>
size_t queueArr<T>::Size() const{
    return q_size;
```

```cpp

template <typename T>
void queueArr<T>::Clear(){
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

template <typename T>
T queueArr<T>::front() const{
    //what will happen if the queue is already empty?
    if(Empty()){
        std::cout<< "The Queue is Empty. \n";
        return T();
    }
    return q_array[q_front];
}

//back
template <typename T>
T queueArr<T>::back() const{
    if(Empty()){
        std::cout<< "The Queue is Empty. \n";
        return T();
    }
    return q_array[q_back];
}

template <typename T>
void queueArr<T>::Enqueue(T value){
    //check if the queue is full
    if(q_size == q_capacity){
        std::cout<< "The Queue is Full. \n";
        return;
    }
    //compute for the q_back for circular (use the modulo operator)
    q_back = (q_back + 1) % static_cast<int>(q_capacity);
    //assign the q_array with the updated q_back to the value
    q_array[q_back] = value;
    //increment the q_size
    q_size++;
    return;
}

template <typename T>
void queueArr<T>::Dequeue(){
    //check if it is empty
    if(Empty()){
        std::cout<< "The Queue is Empty. \n";
        return;
    }
    //move q_front circularly
    q_front = (q_front + 1) % static_cast<int>(q_capacity);

    //decrement the q_size
```

```
112     //decrement the q_size
113     q_size--;
114 
115     //reset index when queue becomes empty
116     if(Empty()){
117         Clear();
118         return;
119     }
120 }
121 
122 //copy constructor: deep copy
123 template <typename T>
124 queueArr<T>::queueArr(const queueArr& other){
125     q_capacity = other.q_capacity;
126     q_size = other.q_size;
127     q_front = other.q_front;
128     q_back = other.q_back;
129     q_array = new T[q_capacity];
130     for(size_t i = 0; i < q_capacity; i++){
131         q_array[i] = other.q_array[i];
132     }
133 }
134 
135 //copy assignment operator: deep copy with self-assignment check
136 template <typename T>
137 queueArr<T>& queueArr<T>::operator=(const queueArr& other){
138     if(this == &other) return *this;   // guard against self-assignment
139 
140     delete[] q_array;                  // free existing memory
141 
142     q_capacity = other.q_capacity;
143     q_size = other.q_size;
144     q_front = other.q_front;
145     q_back = other.q_back;
146     q_array = new T[q_capacity];
147     for(size_t i = 0; i < q_capacity; i++){
148         q_array[i] = other.q_array[i];
149     }
150     return *this;
151 }
152 
153 template <typename T>
154 queueArr<T>::~queueArr(){
155     delete[] q_array;
156 }
157 
158 #endif //QUEUEARR_H
```

OUTPUT:

```
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\HOA5\ILOA> g++ -g main.cpp -o main.exe
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\HOA5\ILOA> .\main.exe
front: 1 back: 1
front: 1 back: 3
1 2 3
front: 2 back: 3
front: 3 back: 3
The queue is empty. Nothing to dequeue.
The queue is empty.
front: 1 back: 1 size: 1
front: 1 back: 2 size: 2
front: 1 back: 3 size: 3
front: 1 back: 4 size: 4
front: 1 back: 5 size: 5
The Queue is Full.
front: 1 back: 5 size: 5
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues

Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office,
usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected
to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to
complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs
to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for
each. From the get-go: You must choose whether you are making a linked list or array implementation.

You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the
number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added
so far.

3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic
strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is
processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

MAIN.CPP

main.cpp > main()

```cpp
#include <iostream>
#include "queueLL.h"

int main() {

    Printer printer;

    std::cout << "===== Morning: initial jobs come in =====\n";
    printer.AddJob(101, "Wrio", 12);
    printer.AddJob(102, "Carlo", 5);
    printer.AddJob(103, "Wrenna", 8);

    printer.DisplayJobs();

    std::cout << "\n===== Printer starts working, one job at a time =====\n";
    printer.ProcessNextJob();   // prints Wrio's job

    std::cout << "\n----- Meanwhile, more people submit jobs -----\n";
    printer.AddJob(104, "Rhy", 20);
    printer.AddJob("Walurich", 4);

    printer.DisplayJobs();

    std::cout << "\n===== Printer keeps working -----\n";
    printer.ProcessNextJob();
    printer.ProcessNextJob();

    std::cout << "\n----- One more late submission arrives -----\n";
    printer.AddJob("Late Submitter", 2);

    std::cout << "\n===== Printer finishes off everything remaining =====\n";
    printer.ProcessAllJobs();

    printer.DisplayJobs();

    ExplainDesignChoice();

    return 0;
}
```

Linklist:

queueLL.h > ...

```
#ifndef QUEUELL_H
#define QUEUELL_H

#include <iostream>
#include <string>

// Job Class
class Job {
public:
    int jobID;
    std::string userName;
    int pages;

    Job() {
        jobID = 0;
        userName = "";
        pages = 0;
    }

    Job(int id, const std::string& user, int p) {
        jobID = id;
        userName = user;
        pages = p;
    }
};


// Display Job

inline std::ostream& operator<<(std::ostream& out, const Job& job) {
    out << "Job ID: " << job.jobID
        << " | User: " << job.userName
        << " | Pages: " << job.pages;
    return out;
}


// Queue Node

template <typename T>
class qNode {
public:
    T data;
    qNode<T>* next;
};


// Create Node

template <typename T>
qNode<T>* new_node(const T& newData) {

    qNode<T>* newNode = new qNode<T>;

    newNode->data = newData;
    newNode->next = nullptr;

```

```
58      return newNode;
59  }
60
61
62  // Enqueue
63
64  template <typename T>
65  void enqueue(const T& newData, qNode<T>** frontPtr, qNode<T>** backPtr) {
66
67      qNode<T>* newNode = new_node(newData);
68
69      if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
70          (*frontPtr) = newNode;
71          (*backPtr) = newNode;
72          return;
73      }
74
75      (*backPtr)->next = newNode;
76      (*backPtr) = newNode;
77  }
78
79
80  // Dequeue
81
82  template <typename T>
83  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
84
85      if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
86          std::cout << "Queue is empty." << std::endl;
87          return;
88      }
89
90      qNode<T>* temp = (*frontPtr);
91
92      if ((*frontPtr) == (*backPtr)) {
93          (*frontPtr) = nullptr;
94          (*backPtr) = nullptr;
95          delete temp;
96          return;
97      }
98
99      (*frontPtr) = (*frontPtr)->next;
100     delete temp;
101 }
102
103
104 // Display Queue
105
106 template <typename T>
107 void displayQueue(qNode<T>* frontPtr) {
108
109     if (frontPtr == nullptr) {
110         std::cout << "Queue is empty." << std::endl;
```

```
112     }
113
114     qNode<T>* current = frontPtr;
115
116     while (current != nullptr) {
117         std::cout << current->data << std::endl;
118         current = current->next;
119     }
120 }
121
122 //=======================
123 // Printer Class
124 //=======================
125 class Printer {
126
127 private:
128     qNode<Job>* front;
129     qNode<Job>* back;
130     int pendingCount;
131     int nextAutoID;
132 public:
133
134     Printer() {
135         front = nullptr;
136         back = nullptr;
137         pendingCount = 0;
138         nextAutoID = 1;
139     }
140
141     Printer(const Printer&) = delete;
142     Printer& operator=(const Printer&) = delete;
143
144     // Destructor
145     ~Printer() {
146         while (front != nullptr) {
147             dequeue(&front, &back);
148         }
149     }
150
151     bool IsEmpty() const {
152         return front == nullptr;
153     }
154
155     int PendingCount() const {
156         return pendingCount;
157     }
158
159     void AddJob(int id, const std::string& user, int pages) {
160         Job newJob(id, user, pages);
161         enqueue(newJob, &front, &back);
162         pendingCount++;
163
164         std::cout << "[SUBMIT] " << user
165                   << " submitted Job " << id
```

```
                  << " submitted Job " << id
                  << " (" << pages << " pages)" << std::endl;
    }

    void AddJob(const std::string& user, int pages) {
        AddJob(nextAutoID++, user, pages);
    }

    void DisplayJobs() const {
        if (front == nullptr) {
            std::cout << "\nNo pending jobs." << std::endl;
            return;
        }

        std::cout << "\n========== Pending Print Jobs (" << pendingCount << ") ==========\n";
        displayQueue(front);
        std::cout << "=========================================\n";
    }


    bool ProcessNextJob() {
        if (front == nullptr) {
            std::cout << "[IDLE] No jobs to process." << std::endl;
            return false;
        }

        Job current = front->data;
        std::cout << "[PRINTING] -> " << current << std::endl;

        dequeue(&front, &back);
        pendingCount--;
        return true;
    }

    // Drains whatever is currently in the queue
    void ProcessAllJobs() {
        if (front == nullptr) {
            std::cout << "\nNo jobs to process." << std::endl;
            return;
        }

        std::cout << "\nProcessing remaining print jobs...\n\n";
        while (front != nullptr) {
            ProcessNextJob();
        }
        std::cout << "\nAll print jobs have been completed." << std::endl;
    }
};

#endif //QUEUELL_H
```

OUTPUT:



```
No pending jobs.
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\HOA5\ILOC> .\main.exe
Wrio submitted Job 101
Carlo submitted Job 102
Wrenna submitted Job 103
Rhy submitted Job 104
Walurich submitted Job 105

========== Pending Print Jobs ==========
Job ID: 101 | User: Wrio | Pages: 12
Job ID: 102 | User: Carlo | Pages: 5
Job ID: 103 | User: Wrenna | Pages: 8
Job ID: 104 | User: Rhy | Pages: 20
Job ID: 105 | User: Walurich | Pages: 4
========================================

Processing Print Jobs...

Printing -> Job ID: 101 | User: Wrio | Pages: 12
Printing -> Job ID: 102 | User: Carlo | Pages: 5
Printing -> Job ID: 103 | User: Wrenna | Pages: 8
Printing -> Job ID: 104 | User: Rhy | Pages: 20
Printing -> Job ID: 105 | User: Walurich | Pages: 4

All print jobs have been completed.

No pending jobs.
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\HOA5\ILOC>
```

I chose a linked list over an array for the printer queue because the number of print jobs coming in isn't something I can predict ahead of time — anyone on the floor can submit a job at any moment, and there's no reasonable fixed limit I could set without either wasting memory on an array that's mostly empty or risking rejecting jobs once it fills up. A linked list lets the queue grow and shrink one node at a time, exactly as jobs are submitted and printed, so I'm never allocating more (or less) than what's actually needed at that moment. It also matches what a real print spooler needs functionally: enqueueing a new job at the back and dequeuing the next one from the front are both O(1) operations with a linked list, since I just update a couple of pointers — no shifting elements over like you'd need with an array-based queue once the front index moves. Given that jobs arrive unpredictably and get processed one at a time in FCFS order, the linked list felt like the more natural and efficient fit for how this system actually behaves.

Output Analysis

The output confirms that the queue is behaving as a true first-come-first-served structure — jobs always print in the exact order they were submitted, never out of sequence, no matter when a new job arrives relative to the printer's progress. That's the whole point of a queue, and the trace shows it holding: Wrio (101) went first because he was first in line, and even though Rhy and Walurich submitted their jobs while the printer was mid-way through processing, they still had to wait behind Carlo and Wrenna, who arrived earlier. This is exactly what I'd expect from enqueue-at-the-back, dequeue-from-the-front logic — the printer never "jumps the line" for anyone, which is the fairness a real print spooler needs. One thing I noticed while reading the output that I didn't fully think through beforehand: Walurich's job printed as "Job 1" and the late submitter's as "Job 2," even though the earlier jobs were numbered 101–104. That's because I added an auto-ID convenience overload that starts its own counter at 1, completely separate from whatever IDs get passed in manually — so the two numbering schemes just don't know about each other. The output isn't wrong (the queue order is still correct), but it's a good example of how a program can behave exactly as coded while still producing something a real user would find confusing, which is a distinction I didn't fully appreciate until I saw it printed out in front of me.

Summary of lessons learned: Building this made the abstract idea of a queue click in a way that just reading about FIFO order never did. I already knew "first in, first out" as a definition, but actually watching jobs submitted mid-processing still wait their turn — instead of me just trusting that the logic works — made it concrete. I also picked up some C++ habits I hadn't taken seriously before, like why a class managing raw pointers needs a destructor and why letting it be copy-constructed by default is dangerous (two Printer objects pointing at, and eventually double-freeing, the same nodes).

Analysis of the procedure: Splitting ProcessAllJobs() into a loop over ProcessNextJob() turned out to be the key design decision — without a "do one job" function, there was no way to realistically interleave new submissions with printing, since the original version could only ever drain the whole queue in one uninterruptible sweep. That one change is what let main.cpp actually simulate the scenario the activity asked for, rather than just faking it with print statements.

Analysis of the supplementary activity: The Job/Printer/queue split forced a cleaner separation of concerns than I would've written on my own — the queue functions don't know or care that they're holding print jobs specifically, and the Printer class doesn't know or care how the queue is implemented underneath. That decoupling is the actual lesson the activity was testing, more than the printer scenario itself.

Conclusion: I think I did solidly on the core requirement — a working, memory-safe, no-STL queue that correctly preserves FCFS order under a realistic interleaved scenario — but the auto-ID collision I found while analyzing my own output is a fair area for improvement. A more careful version would have the auto-ID counter check against (or share state with) manually-assigned IDs so job numbers stay meaningful and non-colliding. It's a small bug, but it's a useful reminder to trace through my own output critically instead of assuming correct-looking code means correct behavior.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: for understanding the key concepts of the codes for array

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.